Microservices based schema design

Design rules:

- Each microservice has its own schema.
- No foreign key crosses service boundaries.
- Cross-service links are stored as plain IDs; not DB joins.

1. Identity & Access Service

Users {
Id PK,
First_name,
last_name
email UNIQUE
password_hash
dob
gender
mobile
role_id **FK -> roles.id** ,
status ENUM(ACTIVE, LOCKED, DISABLED), -- for authentication purpose
created_at,
}

Roles {
id PK
name UNIQUE ENUM(ADMIN, TELLER, BRANCH_MANAGER,
LOAN_OFFICER, AUDITOR, CUSTOMER) ,
description
}

Permissions {
id PK ,
name UNIQUE ,
description
}

Role_permissions {
role_id **FK -> roles.id,**
permission_id **FK -> permissions.id,**
PK (role_id, permission_id)
}

2. **Employee Service**

Employees {
id PK
user_id UNIQUE, stores Identity Service users.id
employee_code UNIQUE
branch_id, stores Branch Service branches.id
designation
reporting_manager_id, self-reference within employee service
joining_date
status
}

Relationships

- Each employee maps to exactly one user account
- Each employee belongs to one branch
- Each employee can report to another employee
- Do not enforce a database FK to users if Identity is a separate microservice.
- Store user_id as a reference only.

3. **Branch Service**

Branch {

id PK
branch_code UNIQUE
name
address
city
state
pincode
ifsc_code UNIQUE

}

4. **Customer Service**

Customers {

cif PK
user_id UNIQUE, **stores Identity Service users.id**
customer_type ENUM(INDIVIDUAL, CORPORATE, NRI)
aadhar_number
kyc_status ENUM(PENDING, VERIFIED, REJECTED)
kyc_verified_at
created_at

}

Customer_address {

id PK
cif **FK -> customers.cif**
address_type ENUM(PERMANENT, CURRENT)
line1
line2
city
state
pincode

}

5. **Document / KYC Service**

Documents {

id PK
cif, stores Customer Service customers.cif
document_type
document_number
issue_date
verification_status ENUM(PENDING, VERIFIED, REJECTED)
verified_by, stores Identity Service users.id

verified_at
remarks
created_at
}

6. **Product Service**

Products {

id PK
product_code UNIQUE
name
product_type ENUM(DEPOSIT, LOAN, CARD, INSURANCE)
interest_rate
min_balance
tenure_months
opening_fee
status ENUM(ACTIVE, DISCONTINUED)
Created_at
Updated_at
}

Optional:

Product_rules {

Id,
Product_id FK,
Rule_type,
Rule_value

}

7. **Accounts Service**

Accounts {

id PK
account_number UNIQUE
branch_id, **stores Branch Service branches.id**
product_id, **stores Product Service products.id**
account_type ENUM(SAVINGS, CURRENT)
status ENUM(OPEN, CLOSED, FROZEN, DORMANT)

available_balance
opened_at
closed_at
currency

}

Account_holders {
id PK
account_id FK -> accounts.id
cif, **stores Customer Service customers.cif**
holder_role ENUM(PRIMARY, JOINT, NOMINEE)
is_primary
created_at
}

/* Sample Scenario: If a savings account is created, then it comes under the deposit product which is primary, if we need a debit card for that account then that 'debit card' comes under CARD product, but it's secondary so link it with the account_id

Secondary_link_products {
Account_id FK,
Product_id, **stores Product Service products.id**
} */

8. **Transaction Service**

Transactions {
id PK
txn_ref UNIQUE
account_id, stores Account Service accounts.id
counter_account_id, -- **optional** stores another accounts.id, represents the other account involved in a transaction. It is primarily useful for internal account-to-account transfers within the same bank.
txn_type ENUM(DEBIT, CREDIT)
amount
txn_mode ENUM(UPI, NEFT, CASH, CHEQUE, CARD)
description

status ENUM(PENDING, SUCCESS, FAILURE)
created_at
created_by, stores Identity Service users.id
}

9. **Ledger Service**

Ledger_entries {
id PK
txn_id, **stores Transaction Service transactions.id**
account_id, **stores Account Service accounts.id**
entry_type ENUM(CREDIT, DEBIT)
amount
posting_date
narration
created_at
}

Cross-service relationship map

Here is the clean logical mapping:

- employees.user_id -> **Identity users.id**
- customers.user_id -> **Identity users.id**
- employees.branch_id -> **Branch branches.id**
- customers.cif -> **Customer Service primary key**
- customer_documents.cif -> **Customer customers.cif**
- accounts.branch_id -> **Branch branches.id**
- accounts.product_id -> **Product products.id**
- account_holders.cif -> **Customer customers.cif**
- transactions.account_id -> **Account accounts.id**
- transactions.created_by -> **Identity users.id**
- ledger_entries.txn_id -> **Transaction transactions.id**
- ledger_entries.account_id -> **Account accounts.id**

These are **service-level references**, not cross-database foreign keys.